

Operating Systems — Midterm Practice Exam

Name: _____ Student ID: _____ Date: _____

Total: 100 points. Time allowed: 90 minutes. For code fill-in questions, write your answers in the blank table provided beneath each program. Do **not** modify the given code skeleton.

Q1 Interrupts & I/O (25 pts)

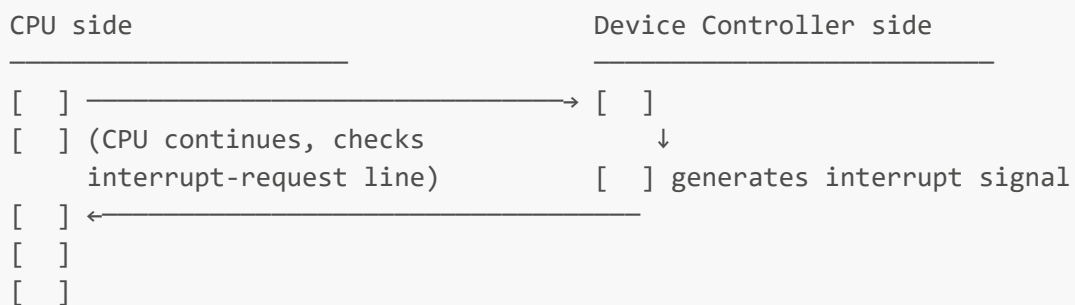
(a) Show the advantage of interrupt-based operations over busy-waiting. (5 pts)

Answer:

(b) The interrupt-driven I/O cycle consists of the following steps. Organize the steps with the flowchart below. (10 pts)

- (i) Input ready, output complete, or error generates interrupt signal
- (ii) Interrupt handler processes data, return from interrupt
- (iii) CPU executing, checks for interrupts between instructions
- (iv) Device driver initiates I/O
- (v) Controller initiates I/O
- (vi) CPU resumes processing of interrupted process
- (vii) CPU receiving interrupt, transfer control to interrupt handler

Fill in each box with the step number (i) – (vii):



(c) What role do device controllers and device drivers play in a computer system? (5 pts)

Device controller:

Device driver:

(d) Explain the benefit of using interrupt vector. **(5 pts)**

Answer:

Q2 System Structures (20 pts)

(a) How does an API function invoke a system call? You **must mention**: software interrupt, dual-mode, system call number (index), and table of code pointer. **(10 pts)**

Answer:

(b) Explain why programmers prefer programming according to API functions rather than invoking system calls directly. **(5 pts)**

Answer:

(c) Describe the pros and cons of the layered operating system structure. **(5 pts)**

Pros:

Cons:

Q3 Processes (20 pts)

(a) Show the state of a process in the following situations: (8 pts)

#	Situation	State
i	After it is created (forked) by a parent process	
ii	Before it is selected by a short-term scheduler	
iii	After it is selected by a short-term scheduler	
iv	Before it invokes a blocking system call	
v	After it invokes a blocking system call	
vi	After the blocking system call is complete	
vii	After it is interrupted by a hardware interrupt	
viii	After its time slice (time quantum) has expired	

(b) Consider the following program. `value` is a **global** variable. (12 pts)

```
int value = 5;

int main(int argc, char *argv[]) {
    pid_t pid;
    pid = fork();

    if (pid > 0) {
        value += 15;
        wait(NULL);
        printf("value = %d\n", value);
    } else if (pid == 0) {
        value += 5;
        printf("value = %d\n", value);
    }
    return 0;
}
```

(i) What is the output of this program? Indicate which process (parent or child) produces each line of output. (6 pts)

Output:

(Explanation:)

(ii) What would happen if the `wait(NULL)` statement were removed? **(6 pts)**

Answer:

Q4 Multithreaded Programming (20 pts)

(a) Compare and contrast the many-to-one and one-to-one thread models. **(8 pts)**

Many-to-one:

One-to-one:

(b) What is the problem of the many-to-many thread model that scheduler activation tries to resolve? **(4 pts)**

Answer:

(c) Describe the operation of scheduler activation. **(8 pts)**

Answer:

Q5 Code Fill-in (15 pts)

Part A — Shared Memory (8 pts)

Complete the following program that calculates the equation $2x + 3y - 5$, where x and y are provided via command-line arguments. The program forks **two child processes** that use shared memory to pass results back to the parent.

```
#include <sys/shm.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);
    int i;
    pid_t pid;
    const int sh_size = 4;

    int segment_id = __A__(IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);

    int *shared_memory = (int *) __B__(segment_id, NULL, 0);

    *shared_memory = __C__;

    for (i = 2; i > 0; i--) {
        pid = __D__();
        if (pid < 0) {
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == 0) {
            if (i == 2) *shared_memory += __E__;
            if (i == 1) *shared_memory += __F__;
            shmdt(shared_memory);
            exit(0);
        } else {
            __G__(NULL);
        }
    }

    *shared_memory -= 5;
    printf("the result is %d\n", *shared_memory);
    shmdt(shared_memory);
    __H__(segment_id, IPC_RMID, NULL);
    return 0;
}
```

Blank	Answer
(A)	
(B)	
(C)	
(D)	
(E)	
(F)	
(G)	
(H)	

Part B — Shell Interface (7 pts)

Complete the following C++ program which serves as a simple shell interface. The shell accepts user commands and then executes each command in a **separate process**. The shell allows the child process to run in the **background** if the command ends with `&`.

```
#include <iostream>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
using namespace std;

int main(void) {
    char inputBuffer[80];
    bool background;

    while (cin >> inputBuffer) {
        cout << endl << " COMMAND->";
        pid_t pid;

        if (inputBuffer[strlen(inputBuffer)-1] == '&') {
            background = __(A)__;
            inputBuffer[strlen(inputBuffer)-1] = '\\0';
        } else {
            background = __(B)__;
        }

        pid = __(C)__( );

        if (pid == 0) {
            __(D)__(inputBuffer);
        } else if (pid > 0) {
            if (background == __(E)__)
                __(F)__(NULL);
        }
    }
}
```

```
    }
    return 0;
}
```

Blank	Answer
(A)	
(B)	
(C)	
(D)	
(E)	
(F)	

Bonus CPU Scheduling (+10 pts)

Consider processes **P1** and **P2** with the following characteristics:

- **P1**: arrives at $t = 0$; CPU burst = 8 ms; I/O = 4 ms; CPU burst = 8 ms
- **P2**: arrives at $t = 2$; CPU burst = 2 ms; I/O = 2 ms; CPU burst = 16 ms

Use a **preemptive multilevel feedback queue** scheduler with the following queues (higher-numbered queue has higher priority; a process entering the ready queue can preempt a running process in a lower-priority queue):

- **Q2** (highest): Round-Robin, time quantum = 4 ms
- **Q1**: Round-Robin, time quantum = 6 ms
- **Q0** (lowest): FCFS

A process that uses its full quantum is moved to the next lower queue. A process returning from I/O re-enters **Q2**.

(a) Draw the Gantt chart. (6 pts)

Time (ms):

02468101214161820222426

28303234

Queue:

(b) Calculate the waiting time for each process and the average waiting time. (4 pts)

Process	Arrival	Finish	Turnaround Time	Waiting Time
P1	0			
P2	2			

Average Waiting Time = _____ ms

End of exam. Good luck!